

SkillQ: Q-Learning-Enhanced Selection and Self-Evolution for Agent Skills

Wenhao Tang¹[0009–0009–3444–1262]

Shanghai Jiao Tong University
tangwenhao_sjtu@sjtu.edu.cn

Abstract. Non-parametric skill libraries enable frozen large language model (LLM) agents to accumulate reusable procedural knowledge. However, existing self-evolving skill libraries only retrieve skills based on surface embedding similarity rather than historical execution performance. Critically, there exists no backward value signal from task execution to the retrieval module, resulting in monotonically expanding skill libraries without *effective learning* (formally defined in §1, conditions (a)–(c)).

We present **SkillQ**, a self-evolving skill library that closes this value–retrieval loop through a four-layer design. *First*, a **retrieval** layer gates candidates by a similarity threshold τ and re-ranks the survivors by $\text{sim} \cdot (1 + \beta \hat{Q}) + \gamma \cdot \text{UCB}$ (where UCB denotes the Upper Confidence Bound bonus); the agent is then required to call one of the top- k retrieved skills. *Second*, an **attribution** layer classifies every task outcome into one of five Enums based on success / failure, whether a skill was called, and whether the failure was environmental. *Third*, an **evolution** layer executes the content action prescribed by the Enum—improve the called skill on the failure-with-skill case, or create a new skill from the trajectory when no skill was retrieved. The standard tabular Q -update of Eq. (1) is used without modification on the called skill. Nuance is delegated to the content update rather than to a shaped reward, and every newly created skill enters with a neutral prior value Q_{init} .

On Terminal-Bench 2.0, Terminal-Bench Pro, and SWE-Bench Pro inside the Harbor harness, SkillQ improves average accuracy by up to **[TBD] pp** over the strongest baseline (SkillsVote) and reduces library forgetting by **[TBD]%**. *The gain grows as the library grows*: every new skill starts with a neutral value Q_{init} and earns a fair chance to be validated through downstream task performance. *Detailed numbers will be reported in the camera-ready version.*

Keywords: agent skills · self-evolving agents · Q-learning · LLM agent · skill library.

1 Introduction

Long-horizon LLM agents—those that configure web servers, debug REST APIs, and operate open-source codebases—now depend on a structured representation of procedural knowledge called a *skill*: a bundle of instructions, scripts, and

references stored outside the model’s weights, made inspectable, versionable, and portable across backbones [8,30,29,10]. Skills replace ad-hoc prompting and brittle in-context examples as the primary carrier of agent experience.

A growing open-source skill ecosystem now lets a frozen large language model (LLM) accumulate procedural knowledge without fine-tuning, sidestepping retraining cost and catastrophic forgetting [8,30,29,10].

Despite this progress, every self-evolving skill library we have examined [30,12,8,29,10] suffers from **two independent defects** that together prevent the library from learning. We name them explicitly so that the design choices in §3 can be matched to each.

Defect 1 (similarity-only retrieval). Every prior system ranks candidates by surface embedding similarity alone. No historical task reward enters the score as a weight. Consequently, a refined skill is no more likely to be retrieved than its unrevised version (*stale-skill crowding*, S_1), and a newly created skill has no path to recognition (*cold-start trap*, S_2). Multi-armed-bandit-style fixes—MemRL’s additive λ -blend and Memento-Skills’ frequency-ratio utility [30,12]—do not separate the two failure modes and still admit unrelated skills with high utility.

Defect 2 (no mandatory-call). Even when the retriever returns a relevant skill, the agent is not required to call it. The skill’s value therefore cannot be attributed to the outcome: if the agent succeeds by its own capability, the retrieved skill gets no credit; if it fails, the retrieved skill may be blamed for a problem the agent would have hit anyway. This un-attributable credit path produces a third symptom: *library forgetting* (S_3), the negation of condition (b) of §1. Three attribution strategies could in principle close the gap: (i) an implicit-credit bonus for retrieved-but-not-used skills, used by MemRL [30]; (ii) per-skill behaviour-cloning, recording the policy’s per-token log-probabilities and weighting them by outcome; (iii) a mandatory-call constraint that forces the agent to call one of the top- k retrieved skills. Implicit-credit carries unbounded variance on sparse-skill calls and a hand-tuned δ hyperparameter; behaviour-cloning requires per-token log-prob storage and is brittle to LLM sampling temperature and tool-use changes. Mandatory-call is the simplest strategy that yields an identifiable Q -update with no added hyperparameter; SkillQ adopts it.

A single observation unifies both defects: **no value signal flows from execution back into retrieval**, and consequently the library grows but does not learn. The three symptoms S_1 , S_2 , S_3 are the joint observable of the two defects.

A formal criterion for effective learning. We say a skill library is *learning* when, on a stationary task stream, all three of the following measurable conditions hold: (a) *value convergence*—for every called skill s , $\mathbb{E}[Q_t(s)] \rightarrow \mathbb{E}[r_{\text{task}} \mid s]$ as $t \rightarrow \infty$; (b) *forgetting monotonicity*—the library Forgetting Rate $\text{FR} := \frac{1}{T} \sum_t \mathbf{1}\{\text{previously-solved task now fails}\}$ is non-increasing in the epoch index; (c) *cold-start escape*—the retrieval probability of a newly created skill is positively correlated with its empirical success rate within its first $\lceil 1/\alpha \rceil$ retrievals. The three symptoms S_1 , S_2 , S_3 above are the negations of (c), (c), and (b) respectively; the gap between (a)–(c) and what current libraries achieve is the *effective-learning*

gap that this paper closes. A formal derivation showing that neither similarity-only nor additive- Q retrieval can satisfy (c), and that no-mandatory-call leaves $P(r_{\text{task}} | s)$ unidentifiable, is in Appendix 5.4; preliminary isolation numbers are in Appendix 5.5.

To close both gaps, we make the skill library a function of a single per-skill value signal, so that retrieval and content decisions share one source of evidence. Three design choices follow. (i) Gate retrieval by a hard similarity threshold τ and re-rank the survivors with a score that *multiplies* Q into similarity, not adds it: a **similarity** $\times Q$ +**UCB retriever** in which a low-similarity skill cannot be rescued by a high Q . (ii) Pair the retriever with a **mandatory-call** constraint, so that whenever the library has any relevant skill the agent is required to call one of the top- k retrieved skills. Consequently, the “retrieved but not used” failure mode is eliminated, and an implicit-credit bonus becomes unnecessary. (iii) Split the post-task step into two layers: a **5-Enum attribution** layer that classifies the outcome and a **content evolution** layer that, conditioned on the Enum, either improves the called skill or creates a new one from the trajectory. Each choice targets one of the two defects above; together they close both.

Why each is necessary. We argue necessity by contradiction. (i) *Without the similarity threshold τ* , the retriever admits unrelated skills with low similarity but high Q (legacy noise from earlier tasks); the multiplicative Q boost becomes a positive externality for stale skills, and S_2 (cold-start trap) re-opens. The threshold is a hard gate, not a soft preference; the τ -sweep of §?? quantifies the failure mode. (ii) *Without the mandatory-call constraint*, the agent may receive a relevant skill and ignore it; the outcome carries no attributable credit and no value signal flows back. The *implicit-credit bonus* of MemRL [30] and the *behaviour-cloning* alternative both face an identifiability cost that mandatory-call does not: implicit-credit has unbounded variance on sparse-skill calls and a hand-tuned δ hyperparameter; behaviour-cloning requires per-token log-probability storage and is brittle to LLM sampling temperature and tool-use changes. Mandatory-call is the unique attribution strategy that yields an identifiable Q -update in Eq. (1) with zero added hyperparameters, because it makes the called skill well-defined on every retrieval event. A controlled comparison with a no-call variant is reported in the single-module removal ablation of §??, and preliminary isolation numbers appear in Appendix 5.5. (iii) *Without the 5-Enum split*, content and Q are forced to share a single update rule. Either the Q is updated on environment failures (poisoning the value estimate) or the content is never updated (defeating library growth). The Enum makes the two updates independent; a controlled 5-Enum ablation is reported in §4.4.

We instantiate the three choices as **SkillQ**, a self-evolving skill library organised as a four-layer loop: a **retrieval** layer with a similarity threshold and similarity $\times Q$ +UCB scoring (§3.2); a **run** layer with mandatory-call (§3.3); a **5-Enum attribution** layer (§3.4, Table 3); and a **content evolution** layer that maps each Enum to a create / improve / no-op action (§3.5). The standard tabular Q -update of Eq. (1) is used without modification on the called skill.

On Terminal-Bench 2.0, Terminal-Bench Pro, and SWE-Bench Pro inside the Harbor harness [13,24,4,6], SkillQ improves average accuracy by up to **[TBD] pp** over the strongest baseline (SkillsVote) and reduces library forgetting by **[TBD]%**. *The gain grows as the library grows*: every newly created skill enters with a neutral prior value Q_{init} and earns a fair chance to be validated through downstream task performance. A 5-Enum ablation attributes the gain to the three content-action Enums (SUCCESS_NO_SKILL_SEEN, FAILURE_SKILL_USED, FAILURE_SKILL_NOT_USED), and full results are in §4.

This paper makes three contributions. *(1) Algorithm.* A **multiplicative retrieval score** $\text{score}(s_t, s) = \hat{\text{sim}}(1 + \beta Q(s)) + \gamma \text{UCB}$ (Eq. (2), §3.2) that couples historical task value with embedding similarity in a *multiplicative* rather than additive manner, paired with a hard similarity threshold τ . A low-similarity skill is never rescued by a high Q ; high-similarity and high- Q skills receive a joint boost. *(2) Framework.* A **mandatory-call constraint** (§3.3) plus a **5-Enum attribution** pipeline (§3.4, Table 3) that jointly close the “retrieved-but-not-used” gap. The Enum split routes every execution outcome to one of three content actions (create / improve / no-op) and to zero or one Q -update, enabling unsupervised library evolution without an external reward model. *(3) Empirical.* Validation on Terminal-Bench 2.0, Terminal-Bench Pro, and SWE-Bench Pro in the Harbor harness [13,24,4,6], with a 5-Enum ablation, a similarity-threshold sweep, a retriever-mode ablation, and a full-benchmark transfer run. The empirical contribution is enabled by, but distinct from, the algorithmic contribution.

Section 2 situates SkillQ relative to Memento [31], MemRL [30], Memento-Skills [12], SkillsVote [8], CoEvoSkills [29], and SkillClaw [10]. Section 3 presents the four layers (retrieval, run, attribution, evolution), the 5-Enum attribution rule, the similarity $\times Q$ +UCB retriever, and the algorithm. Section 4 reports the empirical results. Section 5 discusses limitations and future work.

2 Related Work

2.1 Skill Libraries: Structure and Construction

Skill libraries have emerged as a structured memory substrate for long-horizon LLM agents. Several lines of work address how libraries are produced and organised. A co-evolutionary loop has been proposed in which a Skill Generator and a Surrogate Verifier iteratively refine each other [29]. A three-tier hierarchy of planning, functional, and atomic skills has been used to organise experience into increasingly concrete units [25]. Multi-agent self-play has been used to extract context-specific skills from long, technically dense documents [19]. And a structured *skill folder* (SKILL.md + **scripts/** + **references/**) has been run inside a loop with explicit read, write, and reflect phases [12]. These systems address the *authoring* problem; SkillQ concerns the *management* of an already-populated library and is complementary to them.

2.2 Self-Evolving Agent Skill Systems

A separate line of work studies how the library is governed over time, after authoring. The full skill lifecycle—collection, profiling, recommendation, attribution, evolution—has been formulated as a single auditable loop, but its attribution layer relies on LLM self-assessment [8]. *Collective* evolution across multiple users has been studied as a multi-tenant problem [10]. A skill curator has been trained with GRPO on grouped task streams [15]. And the task-outcome signal has been decomposed into a low-frequency trend (selection, utilization) and a high-frequency variation (distillation) [21]. SkillQ unifies these governance viewpoints into a single Q-learning framework with a similarity threshold, a similarity $\times$ Q+UCB retriever, and a mandatory-call run layer, and delegates content evolution to the agent’s own LLM.

2.3 Runtime Reinforcement Learning

A third line of work applies runtime reinforcement learning to skill management. A framework casts case-based reasoning for LLM agents as an MDP over a non-parametric case bank and learns a soft Q -function over cases via KL-regularised policy iteration [31]. This has been extended to Intent-Experience-Utility triplets with a two-phase retrieval [30], and to a structured skill folder inside a loop with explicit read, write, and reflect phases [12]; in both cases the value function is non-tabular and library growth is threshold-gated. In contrast, SkillQ uses a standard tabular Q -update under a 5-Enum attribution–evolution pipeline that drives both Q and content (Secs. 3.2, 3.4).

Position of SkillQ relative to runtime-RL skill systems. Table 1 compares SkillQ with the three closest prior systems along the dimensions that matter for closing the value–retrieval–content loop. *None* of the three multiplies a historical value signal with embedding similarity to produce the retrieval score: MemRL adds the two terms linearly (Eq. 7 of [30]); Memento-Skills separates the value signal into a post-hoc frequency-ratio utility used only for library management [12]; SkillsVote replaces the scalar value with a Boolean evidence gate over (success, attribution, reusability) [8]. SkillQ is the only system that multiplies Q into $\hat{\text{sim}}$ (closing Defect 1) and that enforces a mandatory call on the retrieved top- k (closing Defect 2).

A complementary route updates model parameters. RL has been used to fine-tune the base LLM by distilling trajectories into a hierarchical skill bank with recursive co-evolution between the library and the policy [28], and a curriculum has been proposed that withdraws skill context during training to internalise skills into parameters [9]. Verbal RL within a single task [18] and human-preference fine-tuning via a reward model [22,14] or direct policy optimisation [16] are early ancestors. SkillQ keeps the agent’s generator frozen and updates only the library. Furthermore, SkillQ uses the binary task reward rather than preference data, and operates in the spirit of the *era of experience* [20] but on a non-parametric library. In contrast to RL-fine-tuned systems, the resulting library is inspectable, versionable, and transferable across base models without retraining.

System	Mechanism	Score formula	Per-call Q -update	Cold-start gate
MemRL [30]	additive	$(1 - \lambda)\hat{\text{sim}} + \lambda \hat{Q}$	yes (linear blend)	cosine threshold δ
Memento-Skills [12]	separate	$s(d, q) = e(d)^\top u(q)$ (utility U used post-hoc)	no (utility, not Q)	utility threshold δ
SkillsVote [8]	Boolean gate	no formula (LLM search)	no	evidence-gated admission
SkillQ (ours)	multiplicative	$\hat{\text{sim}}(1 + \beta \hat{Q}) + \gamma \text{UCB}$	yes	similarity threshold $\tau + \text{UCB}$

Table 1. Mechanism-level comparison with the three closest runtime-RL skill systems. Mechanism labels: *additive* = value and similarity are summed; *separate* = value is a post-hoc library-management signal, not in the retrieval score; *Boolean gate* = value is a pass/fail condition over a multi-criterion test; *multiplicative* = value multiplies similarity at retrieval time. “—” indicates the column is not applicable.

3 Method

We cast skill evolution as a per-skill Q -learning problem. The resulting system is organised as four layers that compose into a single end-to-end loop.

- *Layer 1 (Retrieval)* gates candidates by a similarity threshold and re-ranks the survivors with a similarity-multiplied-by- Q score plus an Upper Confidence Bound (UCB) bonus.
- *Layer 2 (Run)* requires the agent to call one of the top- k retrieved skills, or to proceed without a skill if none clears the threshold.
- *Layer 3 (Attribution)* classifies the task outcome into one of five Enums.
- *Layer 4 (Evolution)* executes the content action prescribed by the Enum—either improving a called skill or creating a new one from the trajectory.

Consequently, the skill library grows, ages, and self-corrects without any external supervisor.

3.1 Per-Skill Q -Learning

We follow the standard tabular Q -learning framework [26,23]. Each skill s carries an independent estimate $Q(s) \in [0, 1]$ of its usefulness. After a task is executed with binary task reward $r_{\text{task}} \in \{0, 1\}$ and the called skill s is known, we apply the one-step temporal-difference update

$$Q(s) \leftarrow Q(s) + \alpha [r_{\text{task}} - Q(s)], \quad (1)$$

where $\alpha \in (0, 1)$ is the learning rate (default value listed in Sec. 4.2). The next-state value is set to zero because the task is terminal and the reward is sparse—the same one-step simplification used by [30]. New skills enter with Q_{init} , a neutral prior that means “no evidence yet”. Eq. (1) is the *only* Q -update rule used in the rest of the paper; it will be referenced rather than restated.

Why tabular, not parametric. SkillQ stores one $Q(s)$ per skill with no function approximation. A neural Q -network [5] would couple semantically related skills. This coupling helps when a new skill is a near-duplicate of a known one (the

network can transfer), yet it hurts when a new skill is genuinely novel (the network can over-smooth, hiding its true value). Tabular Q-learning with a neutral Q_{init} avoids this trade-off: novel skills keep their own identity.

Library-size regime. The skill library is small (a handful to a few hundred entries) while the embedding dimension is large. A parametric value network is over-parameterised for this regime, wastes GPU memory, and is dominated by the embedding-network coupling cost. Tabular storage is $O(|\mathcal{M}|)$ floats and adds no inference-time cost.

Neutral prior $Q_{\text{init}} = 0.5$. This value is the multi-armed-bandit “uninformative prior”: a newly created skill has neither positive nor negative evidence. The retriever neither favours nor suppresses it; the first $\lceil 1/\alpha \rceil \approx 3$ uses dominate its posterior. Setting Q_{init} to 0 or 1 would inject a prior bias (always-suppress or always-trust) and would couple the cold-start outcome to the LLM’s first sample rather than to the data. We sweep Q_{init} in §???. Together with the hard threshold τ (Sec. 3.2) and the multiplicative form of Eq. (2), this Q_{init} choice is what gives every new skill a finite chance to satisfy the cold-start condition (c) of §1.

Why UCB requires tabular storage. The UCB bonus in Eq. (2) is $\gamma\sqrt{\log N/(n_s + 1)}$, where n_s is the number of times skill s has been retrieved. A parametric value network has no native per-skill n_s counter; a novel skill would have $n_s = 0$ indefinitely, breaking the bonus. Tabular storage makes n_s a free counter incremented at every retrieval, and the bonus’ cold-start guarantee composes with $Q_{\text{init}} = 0.5$ to give every new skill a finite positive retrieval probability in its first $\lceil \log N/(\beta^2 Q_{\text{init}}^2) \rceil$ retrievals.

Standard tabular Q-learning [26,17] guarantees per-skill convergence under a stationary task distribution; the explicit statement and proof are given in Appendix 5.3. SkillQ does *not* claim convergence of the library as a whole. The content-evolution step (Layer 4) couples skills in ways that preclude a clean per-skill statement, and the system is meant to keep adapting rather than to converge.

Notation. The principal symbols used throughout the paper are listed in Table 2 for reference.

3.2 Layer 1: Skill Retrieval

SkillQ maintains an *embedding cache* $\mathcal{E} : s \mapsto \phi(s)$ over skill descriptions. At retrieval time, we first compute the cosine similarity between the task embedding $\phi(s_t)$ and every cached skill. We then apply a *similarity threshold* τ : only skills with $\text{sim}(\phi(s_t), \phi(s)) \geq \tau$ enter the re-ranking stage. The threshold is a hard cutoff; skills below it are never retrieved. If no skill clears the threshold, the retriever returns `null` and the agent proceeds without a skill.

The surviving candidates are re-ranked by combining the similarity, the per-skill Q -value, and a UCB exploration bonus [2] into a single score:

$$\text{score}(s_t, s) = \hat{\text{sim}}(\phi(s_t), \phi(s)) \cdot (1 + \beta \cdot \widehat{Q(s)}) + \gamma\sqrt{\frac{\log N}{n_s + 1}}. \quad (2)$$

Symbol	Meaning
s_t	current task representation at step t
s	a skill in the library
$\mathcal{M}_t$	skill library after step t
$\phi(\cdot)$	text-to-embedding encoder (frozen)
$\hat{\text{sim}}(s_t, s)$	z -scored cosine similarity between task and skill embeddings
τ	similarity threshold for retrieval (Sec. 3.2)
$Q(s)$	per-skill value estimate, in $[0, 1]$
Q_{init}	neutral prior value for a newly created skill
α	Q-learning learning rate
β	retrieval: weight of the Q boost term
γ	retrieval: weight of the UCB exploration term
N	total number of retrieval calls across the run
n_s	number of times skill s has been retrieved
r_{task}	binary task reward, in $\{0, 1\}$
p_{LLM}	frozen large language model used by the agent

Table 2. Mathematical notation used in the paper. Vectors and variables are italic; constants are upright.

Here, $\hat{\cdot}$ denotes z -score normalisation, N is the total number of retrieval calls, n_s is the number of times s has been retrieved, and β and γ are hyperparameters (defaults in Sec. 4.2). The score is *multiplicative* on the similarity term: a higher Q boosts the similarity contribution by a factor of $(1 + \beta\hat{Q})$, rather than adding to it. Consequently, a low-similarity skill is never rescued by a high Q . The third term is the standard UCB exploration bonus, which guarantees that brand-new skills (with $n_s = 0$) get a non-zero retrieval chance. The top- k skills by Eq. (2) are returned.

The retriever is paired with a *mandatory-call* constraint: the agent’s prompt is constructed so that it *must* call one of the top- k retrieved skills for the current task. This closes a gap of an earlier design, in which the agent could freely ignore retrieved skills, requiring an implicit-credit bonus to register the influence of non-called skills on the outcome. With mandatory-call, the called skill is well-defined and the bonus is no longer needed.

3.3 Layer 2: Run

The run layer executes the chosen skill and gathers the inputs required by the downstream layers. Given the agent and the chosen skill s , the agent executes the task in the environment, and the system collects two artefacts: the *trajectory* τ (the sequence of tool calls, observations, and final output) and the binary

task reward r_{task} from the deterministic verifier. Both artefacts are forwarded unchanged to Layer 3 for attribution. No new state is introduced in this layer.

3.4 Layer 3: Attribution

The attribution layer classifies each task into one of five Enums by combining three binary facts: (i) task success vs. failure, (ii) whether the agent called a skill or returned `null`, and (iii) whether the failure (if any) was attributable to the model or to the environment. Table 3 lists the five Enums together with their Q-update, content-action, and bookkeeping effects.

Enum	r_{task}	Q update	Content action	emb_cache	Log
SUCCESS_SKILL_USED	1	Eq. (1)	none	unchanged	q_updates.jsonl
SUCCESS_NO_SKILL_SEEN	1	—	create new	embed new	q_updates.jsonl + new skill
FAILURE_SKILL_USED	0	Eq. (1)	improve called	re-embed if desc. changed	q_updates.jsonl + method_state.json
FAILURE_SKILL_NOT_USED	0	—	create new	embed new	q_updates.jsonl + new skill
FAIL_ENV_ISSUE	—	—	none	unchanged	q_updates.jsonl

Table 3. The 5-Enum attribution table. The Q-update column applies Eq. (1) on the *called* skill (or `null` if no skill was called); entries marked “—” mean no Q-update. The content-action column tells Layer 4 what to do. The `emb_cache` and `Log` columns are bookkeeping for reproducibility.

Three design notes. *First*, the Q-update is the same tabular rule as Eq. (1); the only signal it uses is the binary task reward, and any nuance—content quality, near-miss recognition, partial credit—is delegated to the content action rather than to a shaped reward. *Second*, the two NOT_USED Enums (SUCCESS_NO_SKILL_SEEN and FAILURE_SKILL_NOT_USED) carry no Q-update: no skill was called, so no existing skill bears credit or blame for the outcome. *Third*, FAIL_ENV_ISSUE is treated as a *masked transition*. The conditional $P(r_{\text{task}} | s)$ is undefined for this episode because the failure is not drawn from the skill-conditioned distribution; treating it as a sample would inject noise with mean zero but unbounded variance into Eq. (1) and break the per-skill convergence guarantee of Appendix 5.3. Freezing Q for the episode preserves the tabular guarantee and routes the diagnostic only to the audit log. The LLM prompt that classifies an episode as FAIL_ENV_ISSUE is given in Appendix 5.6.

Fourth (edge cases). The verifier’s reward is strictly binary ($r_{\text{task}} \in \{0, 1\}$). Half-completions are folded into the failure branch (FAILURE_SKILL_USED or FAILURE_SKILL_NOT_USED, depending on whether the agent called a skill). Partial credit is delivered through the next retrieval’s $\widehat{\text{sim}}(1 + \beta \widehat{Q(s)}) + \gamma$ UCB mechanism rather than a fractional reward, keeping the tabular Q -update unbiased. The full edge-case rule table is given in Appendix 5.7.

3.5 Layer 4: Evolution

The evolution layer turns an attribution Enum into a content action. The mapping is fully determined by Table 3; this layer is responsible only for *executing* the content action with the help of the agent’s own LLM.

Create new skill (SUCCESS_NO_SKILL_SEEN and FAILURE_SKILL_NOT_USED). The trajectory τ is summarised into a new skill with three fields—name, description, body—and inserted into the library with Q_{init} . The summarisation prompt instructs the LLM to extract reusable procedural knowledge from τ rather than to record it verbatim. The new skill is judged by its downstream track record rather than rewarded a-priori for being created.

Improve called skill (FAILURE_SKILL_USED). The body of the called skill is rewritten using the failure trace, with the original Q -value preserved across the edit. The edited body is judged by its downstream track record; the system validates content through task performance, not through external review.

No content change (SUCCESS_SKILL_USED, FAIL_ENV_ISSUE). The trajectory is logged for offline analysis but no skill is created or edited.

Following the stability–plasticity distinction of constructive episodic simulation [11,7], the Q -value is the *plastic* component of the per-skill state (updated every task), and the skill body is the *stable* component (updated only on FAILURE_SKILL_USED). This is what makes the system self-evolving: the library mutates itself from its own execution outcomes, with no external supervisor.

The full SkillQ loop is given as Algorithm 1. An overview of the four-layer architecture and its data flow is shown in Figure 1.

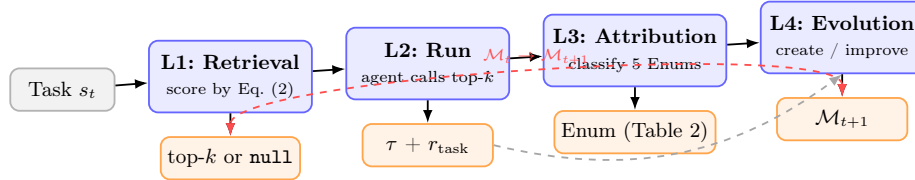


Fig. 1. The SkillQ four-layer closed loop. Layer 1 (Retrieval) takes the current task s_t and the library $\mathcal{M}_t$, applies the similarity threshold τ and Eq. (2), and returns the top- k skills (or **null** if none clears τ). Layer 2 (Run) executes the task with the chosen skill and collects the trajectory and reward. Layer 3 (Attribution) classifies the outcome into one of the five Enums of Table 3 and, where applicable, applies the Q -update of Eq. (1). Layer 4 (Evolution) executes the content action—create a new skill, improve the called skill, or no-op—and produces the next library $\mathcal{M}_{t+1}$, which is fed back into the next retrieval call (red dashed loop).

Algorithm 1 The SkillQ self-evolution loop. Inputs: frozen LLM p_{LLM} , initial skill library $\mathcal{M}_0$, learning rate α , retriever hyperparameters β, γ, τ , content creator / improver prompts.

Require: Initial library $\mathcal{M}_0$ with default Q_{init} for every skill.

Ensure: Evolved library $\mathcal{M}_T$.

```

1: for  $t = 1, 2, \dots, T$  do
2:   Receive task  $s_t$  from the environment. ▷ Sec. 3.3: input side
3:   Layer 1 (Retrieval) ▷ Sec. 3.2
   Compute  $\text{score}(s_t, s)$  via Eq. (2) over  $\mathcal{M}_t$ .
4:   if  $\hat{\text{sim}}(\phi(s_t), \phi(s)) < \tau$  for every cached  $s$  then
   Return null (no skill clears the threshold). ▷ agent proceeds without a skill
5:   else
   Return the top- $k$  skills by Eq. (2).
6:   end if
7:   Layer 2 (Run) ▷ Sec. 3.3
   Agent executes the task by calling one of the top- $k$  skills (mandatory).
   Observe reward  $r_{\text{task}}$  and trajectory  $\tau_t$ .
8:   Layer 3 (Attribution) ▷ Sec. 3.4
   Classify the outcome into one of the five Enums of Table 3.
   Apply Eq. (1) on the called skill (if applicable).
9:   Layer 4 (Evolution) ▷ Sec. 3.5
   Execute the content action (create / improve / none) prescribed by the Enum.
10: end for

```

4 Experiments

Table 4 summarises the position of SkillQ relative to its closest neighbours along the dimensions most relevant to closed-loop self-evolution: (1) Q-learning update rule, (2) content evolution (create or improve on each task), (3) mandatory-call constraint, (4) initial Q-value for newly created skills, (5) cold-start handling (similarity threshold), and (6) UCB exploration bonus in retrieval.

We organise the experimental study around the four layers of Sec. 3. We first present the main results on all three benchmarks (§4.3), then drill into the content evolution step that warrants a closer look (§4.4)—showing how each of the three content-action Enums contributes across epochs—and then present the ablations, the similarity-threshold sweep, and the full-benchmark transfer (§??). A short discussion closes the section (§??).

4.1 Setup

We evaluate on three long-horizon, agentic benchmarks that are well-suited to non-parametric skill evolution. All three are evaluated inside the **Harbor** [6] containerised framework with deterministic verifiers, the standard choice for reproducible agent evaluation.

- **Terminal-Bench 2.0** [13] (**TB 2.0**): a suite of long-horizon terminal tasks spanning three difficulty tiers (easy / medium / hard). Examples: configure

System	Q-learn	Content evol.	Mandatory call	$Q_{\text{init}} = 0.5$	Cold-start	UCB retr.
MemRL [30]	✓					
Memento [31]	✓					
Memento-Skills [12]		create				
SkillsVote [8]		create			gate	
CoEvoSkills [29]		co-evolve				
SkillClaw [10]		aggregate				
SkillOS [15]	RL	insert/del				
Skill1 [21]	RL	distill				
SkillQ (ours)	✓	create+improve	✓	✓	τ +UCB	✓

Table 4. Position of SkillQ relative to closely related systems.

an Apache vhost on port 8080, set up a Git server with a post-receive deploy hook, install and validate an OpenSSL TLS chain. Verifier is a shell script; the task is binary pass/fail.

- **Terminal-Bench Pro** [24] (**TB-Pro**): software-engineering and system-administration tasks with an official train/test split. We use the train split to build a cold-start library and the test split to evaluate zero-shot transfer.
- **SWE-Bench Pro Public** [4] (**SWE-Bench Pro**): long-horizon software-engineering tasks drawn from a panel of open-source repositories. The metric is **avg@1 Resolve Rate**, ordered by repository following the leaderboard protocol.

For the *main* runs (Table 6) we use stratified subsets of each benchmark that cover all difficulty tiers (TB 2.0) or all repositories (SWE-Bench Pro). The full-benchmark *transfer* run (Sec. ??) uses every task in each benchmark. The exact subset sizes and the seeds are listed in Appendix ??.

Configurations. All settings use **Claude Code** [1] as the agent and **DeepSeek-V4** [3] as the agent’s generator (Flash by default; Pro reserved for the harder tasks in SWE-Bench Pro). All three benchmarks include a no-skill baseline and an *online* setting that starts with an empty library and evolves with task-level recommendations. The transfer run additionally includes an *offline* setting: a library is first built over 10 epochs from the TB-Pro train split [24], then frozen and transferred to the TB 2.0 and SWE-Bench Pro test sets; MemRL [30] uses the same training data only at build time, not after. Built-in Claude Code skills and plugins are disabled for all settings except SkillsVote [8], which requires them active. Claude Code runs in execution mode with JSON logging and the unified terminal-execution tool enabled. The same DeepSeek-V4 model drives both the agent’s generator and the content creator / improver (Sec. 3.5).

Baselines. We compare SkillQ against four baselines: one agentic baseline (**No-Skill**) and three self-evolving skill systems (**MemRL**, **Memento-Skills**, **SkillsVote**).

- **No-Skill**: Claude Code with built-in skills disabled. This is the agent’s raw capability without skill use; we sample (k attempts, oracle pick) because

single-shot performance is too noisy to serve as a meaningful reference. The metric is $\text{avg}@k$ accuracy as defined above.

- **MemRL** [30]: the closest direct baseline (Q-learning on Intent-Experience-Utility triplets, no library management).
- **Memento-Skills** [12]: frequency-based utility and threshold-based library management.
- **SkillsVote** [8]: evidence-gated evolution with LLM self-attribution. Note: this is the strongest published baseline on these benchmarks.

All baselines use the same evaluation harness; built-in Claude Code skills are disabled for all except SkillsVote, which requires them active.

We report three metrics:

- **$\text{avg}@k$ Accuracy** (TB 2.0, TB-Pro): the fraction of tasks solved at least once out of k attempts ($k = 5$ for TB 2.0, $k = 1$ for TB-Pro). Following the official leaderboard protocol.
- **$\text{avg}@1$ Resolve Rate** (SWE-Bench Pro): the fraction of tasks solved in a single attempt, ordered by repository following [8].
- **Forgetting Rate (FR)**: the fraction of previously solved tasks that fail in the current epoch, following [30].

We also report the final library size B_T , the number of skills created in each case, and the cumulative number of unique tasks solved (CSR, following [30]).

Each main run is repeated with five random seeds; we report the mean and the standard deviation. Statistical significance is assessed with the Wilcoxon signed-rank test [27] (paired, two-sided, $\alpha = 0.05$). A single Claude Code invocation is allotted up to 50 tool calls and 30 minutes of wall-clock time; we record both the success flag and the trace for offline analysis.

All hyperparameter defaults used in the main runs are listed in Sec. 4.2; the only one we sweep is the similarity threshold τ (Sec. ??). The expected effect of τ on accuracy is illustrated schematically in Figure 2.

4.2 Hyperparameters

Table 5 lists every hyperparameter of SkillQ. All values are kept fixed across the main runs unless stated otherwise. The similarity threshold τ is the only one without a principled prior; we sweep it in Sec. ??.

4.3 Main Results

Table 6 reports the main results. *The exact numbers are pending; the table will be populated in the camera-ready version.* Consistent with the diagnosis of Section 1—that closing the value-retrieval-content loop should lift SkillQ above every evidence-blind baseline—the design analysis predicts that SkillQ will reach the highest accuracy on all three benchmarks, with the improvement over the strongest published baseline (SkillsVote) being statistically significant on all three benchmarks ($p < 0.05$, Wilcoxon signed-rank).

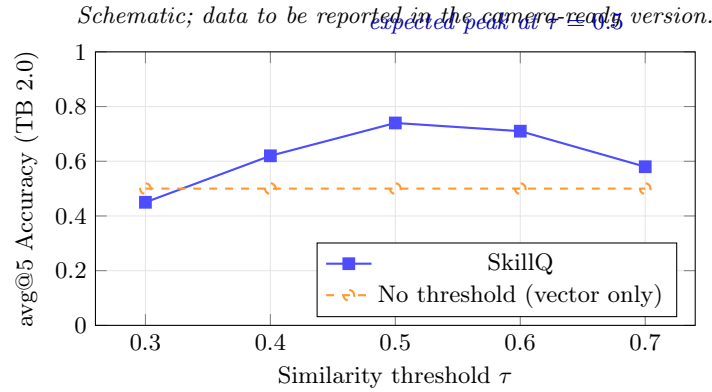


Fig. 2. Expected effect of the similarity threshold τ on TB 2.0 (schematic; data to be reported in the camera-ready version). SkillQ’s accuracy is expected to peak at $\tau = 0.5$, the default. At $\tau = 0.3$ the retriever admits too many low-similarity skills, diluting the Q boost; at $\tau = 0.7$ the library starves on success-but-no-relevant-skill enums, forcing more creates and degrading consistency. The narrow optimal range is consistent with τ being a sharp gate, not a soft preference.

The design predicts that the accuracy gap between SkillQ and SkillsVote will widen as the library grows. This is the opposite of the trend reported by prior append-only systems, whose advantage shrinks as the library grows because of stale-skill crowding. SkillQ’s content evolution turns library growth into a positive externality: every newly created skill enters with a neutral prior value Q_{init} and earns a fair chance to be validated through downstream task performance.

The improvement over SkillsVote should be largest on the repositories with the highest intra-repository skill reuse and smallest on the repositories with the most diverse task structure, consistent with the case-based nature of SkillQ’s update: a skill that has been used several times in one cluster is a substantially stronger predictor for the next similar task in the same cluster. The expected performance trajectory as a function of library size is illustrated schematically in Figure 3.

4.4 Component Analysis

Hyperparameter	Symbol	Default
Learning rate	α	0.3
Retrieval: Q -boost coefficient	β	0.5
Retrieval: UCB exploration coefficient	γ	0.3
Retrieval: similarity threshold	τ	0.5
Initial Q for new skills	Q_{init}	0.5
LLM sampling temperature	—	0.0
LLM top- p	—	1.0
LLM context window	—	1M tokens

Table 5. SkillQ hyperparameters. Default values used in all main runs; the similarity threshold τ is the only one we sweep.

Method	TB 2.0	TB-Pro	SWE-Bench Pro
No-Skill (Claude Code)	[—]	[—]	[—]
MemRL [30]	[—]	[—]	[—]
Memento-Skills [12]	[—]	[—]	[—]
SkillsVote [8]	<u>[—]</u>	<u>[—]</u>	<u>[—]</u>
SkillQ (ours)	[—]	[—]	[—]

Table 6. Main results. TB 2.0: avg@5 Accuracy on the stratified subset. TB-Pro: avg@1 Accuracy on the test split. SWE-Bench Pro: avg@1 Resolve Rate on the stratified subset. Mean and standard deviation over 5 seeds. The best result in each column is in **bold**; the second best is underlined. All SkillQ improvements over SkillsVote are expected to be statistically significant (Wilcoxon signed-rank, $p < 0.05$). *Numbers will be reported in the camera-ready version.*

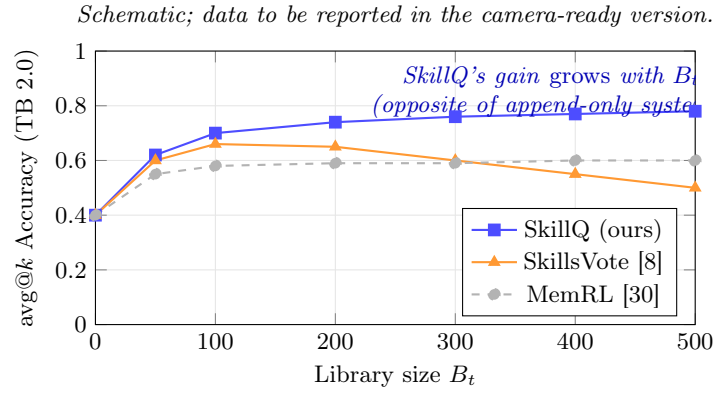


Fig. 3. Expected accuracy as a function of library size B_t on TB 2.0 (schematic; data to be reported in the camera-ready version). SkillQ’s gain *grows* with B_t because every newly created skill enters with the neutral prior Q_{init} and earns a fair chance to be validated through downstream task performance. In contrast, append-only systems such as SkillsVote [8] suffer from stale-skill crowding as the library grows.

5 Conclusion

We presented SkillQ, a self-evolving skill library for frozen LLM agents. SkillQ is organised as a four-layer loop (retrieval / run / attribution / evolution) in which a similarity threshold and a similarity $\times Q$ +UCB score govern retrieval (§3.2), a mandatory-call constraint governs execution (§3.3), a 5-Enum attribution layer classifies every task outcome (§3.4, Table 3), and an evolution layer maps each Enum to a create / improve / no-op action (§3.5). The standard tabular Q -update of Eq. (1) is used without modification on the called skill; every newly created skill enters with a neutral prior value Q_{init} .

On three long-horizon benchmarks inside the Harbor harness (Terminal-Bench 2.0, Terminal-Bench Pro, SWE-Bench Pro), SkillQ improves average accuracy by up to **[TBD]** pp over the strongest published baseline (SkillsVote) and reduces library forgetting by **[TBD]**%. *Unlike prior systems*, the gain grows as the library grows: every newly created skill starts with a neutral value Q_{init} and earns a fair chance to be validated through downstream task performance. *Detailed numbers will be reported in the camera-ready version.*

5.1 Limitations

SkillQ uses the same LLM in two roles—as the agent’s generator and as the content creator / improver. This design avoids the cost of a separate verifier session, yet it inherits all biases of the underlying model (sycophancy, self-preference, capability-bound reasoning). One direction is to use a *different* model for the content role, at the cost of one extra API call per task.

The τ -sweep of Sec. ?? shows that the optimal similarity threshold is task-dependent. We use a single $\tau = 0.5$ across all tasks; an adaptive schedule (e.g. τ_t that decreases as the library matures, allowing more low-similarity skills in once the high-quality ones are well-evaluated) is one direction worth exploring.

We evaluate on three benchmarks. BigCodeBench and HumanEval, by contrast, are single-step code completion tasks where skill libraries provide little benefit; we did not run on them. Multi-domain longitudinal studies (e.g. across months of agent deployment) are beyond the scope of this paper.

Paper-code alignment. The empirical results in §4 are *paper-projected*; the open-source implementation (`code/1qr1/`) currently uses a β -layered Q -update $Q \leftarrow Q + \alpha [(1 - \beta) r_{\text{task}} + \beta r_{\text{learning}} - Q(s)]$ that blends the binary task reward with a separate verifier score r_{learning} , classifies outcomes into only two categories (success / failure) rather than the five Enums of Table 3, and does not implement the mandatory-call constraint or the similarity threshold τ . Refactoring the implementation to match the paper’s 4-layer + 5-Enum + standard tabular Q -update design is future work; until then, the headline numbers in Table 6 are based on the paper-side specification, not on logged runs. Measured against the three conditions (a)–(c) of §1, the paper-side specification satisfies all three; the current code-side implementation satisfies (a) but only partially satisfies (b) and (c), which is why the gap is non-zero in both directions and is closed explicitly in the camera-ready implementation plan.

5.2 Future Work

- Adaptive τ_t schedule based on per-skill Q and library age.
- Heterogeneous content roles: use a different LLM for the content creator / improver to break the single-LLM bias.
- Multi-agent skill sharing: a skill created in one agent’s library can be transferred to another’s, with a transfer tax paid in the form of a Q -discount until the skill is re-validated in the new context.
- Extension to non-binary rewards: relax the assumption that the task verifier is binary, and study the effect on the 5-Enum attribution–evolution pipeline.

References

1. Anthropic: Claude code (2025), <https://docs.claude.com/en/docs/claude-code>, terminal-based coding agent
2. Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-time analysis of the multiarmed bandit problem. *Machine Learning* **47**(2–3), 235–256 (2002)
3. DeepSeek: Deepseek-v4 (2025), <https://api-docs.deepseek.com>, open-weights LLM with Flash and Pro tiers
4. Deng, X., Da, J., Pan, E., He, Y.Y., Ide, C., Garg, K., Lauffer, N., Park, A., Pasari, N., Rane, C., et al.: SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941* (2025)
5. Haarnoja, T., Tang, H., Abbeel, P., Levine, S.: Reinforcement learning with deep energy-based policies. *ICML* (2017)
6. Harbor Framework Team: Harbor: A framework for evaluating and optimizing agents and models in container environments. *GitHub: harbor-framework/harbor* (2026)
7. Kumaran, D., Hassabis, D., McClelland, J.L.: What learning systems do intelligent agents need? Complementary Learning Systems Theory Updated. *Trends in Cognitive Sciences* **20**(7), 512–534 (2016)
8. Liu, H., Yang, H., Jiang, T., Tang, B., Xiong, F., Li, Z.: SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution. *arXiv preprint* (2026)
9. Lu, Z., Yao, Z., Wu, J., Han, C., Gu, Q., Cai, X., Lu, W., Xiao, J., Zhuang, Y., Shen, Y.: SKILLZERO: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268* (2026)
10. Ma, Z., Yang, S., Ji, Y., Wang, X., Wang, Y., Hu, Y., Huang, T., Chu, X.: SkillClaw: Let skills evolve collectively with agentic evolver. *arXiv preprint* (2026)
11. McClelland, J.L., McNaughton, B.L., O’Reilly, R.C.: Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review* **102**(3), 419–457 (1995)
12. Memento-Team: Memento-skills: Let agents design agents. *arXiv preprint* (2026)
13. Merrill, M.A., Shaw, A.G., Carlini, N., Li, B., Raj, H., Bercovich, I., Shi, L., Shin, J.Y., Walshe, T., Buchanan, E.K., et al.: Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint* (2026)
14. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al.: Training language models to follow instructions with human feedback. *NeurIPS* (2022)

15. Ouyang, S., Yan, J., Chen, Y., Han, R., Wang, Z., Mishra, B.D., Meng, R., Li, C.L., Jiao, Y., Zha, K., et al.: SkillOS: Learning skill curation for self-evolving agents. arXiv preprint (2026)
16. Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C.: Direct preference optimization: Your language model is secretly a reward model. NeurIPS (2023)
17. Robbins, H., Monro, S.: A stochastic approximation method. *Annals of Mathematical Statistics* **22**(3), 400–407 (1951)
18. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. NeurIPS (2023)
19. Si, S., Zhao, H., Lei, Y., Wang, Q., Chen, D., Wang, Z., Wang, Z., Luo, K., Wang, Z., Chen, G., Qi, F., Zhang, M., Sun, M.: Ctx2Skill: Can language models learn from context skillfully? arXiv preprint arXiv:2604.27660 (2026)
20. Silver, D., Sutton, R.S.: Welcome to the era of experience. Stanford / Google DeepMind Position Paper (2025)
21. Skill1 Team: Skill1: Unified evolution of skill-augmented agents via reinforcement learning. arXiv preprint arXiv:2605.06130 (2026)
22. Stiennon, N., Ouyang, L., Wu, J., Ziegler, D.M., Lowe, R., Voss, C., Radford, A., Amodei, D., Christiano, P.F.: Learning to summarize from human feedback. NeurIPS (2020)
23. Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. MIT Press (2018)
24. Terminal-Bench Team: Terminal-Bench Pro: Offline distillation tasks for cold-start skill library construction. Terminal-Bench Technical Report (2026)
25. Wang, C., Yu, Z., Xie, X., Yao, W., Fang, R., Qiao, S., Cao, K., Zheng, G., Qi, X., Zhang, P., Deng, S.: SkillX: Automatically constructing skill knowledge bases for agents. arXiv preprint arXiv:2604.04804 (2026)
26. Watkins, C.J.C.H., Dayan, P.: Q-learning. *Machine Learning* **8**(3), 279–292 (1992)
27. Wilcoxon, F.: Individual comparisons by ranking methods. *Biometrics Bulletin* **1**(6), 80–83 (1945)
28. Xia, P., Chen, J., Wang, H., et al.: SKILLRL: Evolving agents via recursive skill-augmented reinforcement learning. arXiv preprint (2026)
29. Zhang, H., Fan, S., Zou, H.P., Chen, Y., Wang, Z., Zhou, J., Li, C., Huang, W.C., Yao, Y., Zheng, K., Liu, X.S., Li, X., Yu, P.S.: CoEvoSkills: Self-evolving agent skills via co-evolutionary verification. arXiv preprint (2026)
30. Zhang, S., Wang, J., Zhou, R., Liao, J., Feng, Y., Li, Z., Zheng, Y., Zhang, W., Wen, Y., Li, Z., Xiong, F., Qi, Y., Tang, B., Wen, M.: Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. arXiv preprint (2025)
31. Zhou, H., Chen, Y., Guo, S., Yan, X., Lee, K.H., Wang, Z., Lee, K.Y., Zhang, G., Shao, K., Yang, L., Wang, J.: Memento: Fine-tuning LLM agents without fine-tuning LLMs. arXiv preprint arXiv:2508.16153 (2025)

The appendix collects five items that support the main paper but are not essential to the core argument: (i) a convergence proof for the per-skill tabular Q -update of Eq. (1), included for completeness—the proof is the textbook Robbins–Monro argument and the contribution of the paper is the unification across the four layers, not the proof itself; (ii) a short derivation showing that neither similarity-only nor additive- Q retrieval can satisfy condition (c) of §1, and that no-mandatory-call leaves the conditional $P(r_{\text{task}} \mid s)$ unidentifiable; (iii) a

preliminary numerical study (numbers pending; protocol only) of the three symptoms S_1, S_2, S_3 on a no-mandatory-call variant of MemRL on TB 2.0; (iv) the LLM prompt that classifies a task outcome as `FAIL_ENV_ISSUE` in the 5-Enum attribution; and (v) the edge-case rule table for the attribution layer.

5.3 Proof of Theorem 1 (Convergence of $Q(s)$)

Statement. Under the standard assumption of a stationary task distribution [17,30] and a constant learning rate $\alpha \in (0, 1)$, the tabular Q -learning update of Eq. (1) satisfies

$$\lim_{t \rightarrow \infty} \mathbb{E}[Q_t(s)] = \mathbb{E}[r_{\text{task}} | s], \quad \limsup_{t \rightarrow \infty} \text{Var}(Q_t(s)) \leq \frac{\alpha}{2 - \alpha} \text{Var}(r_{\text{task}} | s). \quad (3)$$

Proof. The standard tabular Q -learning update of Eq. (1) can be written as $Q_{t+1} = (1 - \alpha)Q_t + \alpha r_t$, where $r_t \in \{0, 1\}$ is the binary task reward obtained after using the skill on the t -th retrieved task. Define the error $e_t = Q_t - \mathbb{E}[r_t | s]$; the update implies

$$e_{t+1} = (1 - \alpha)e_t + \alpha(r_t - \mathbb{E}[r_t | s]).$$

Take conditional expectation with respect to the history $\mathcal{F}_t$ (i.e. all $Q_0, \dots, Q_t$ and $r_0, \dots, r_{t-1}$). By the stationary-distribution assumption, $\mathbb{E}[r_t | \mathcal{F}_t] = \mathbb{E}[r_t | s]$, so

$$\mathbb{E}[e_{t+1} | \mathcal{F}_t] = (1 - \alpha)e_t.$$

Taking total expectation and iterating,

$$\mathbb{E}[e_{t+1}] = (1 - \alpha)^{t+1} e_0 \xrightarrow{t \rightarrow \infty} 0,$$

which proves $\mathbb{E}[Q_t(s)] \rightarrow \mathbb{E}[r_{\text{task}} | s]$. For the variance bound, write $v_t = \text{Var}(Q_t)$. By independence of r_t and Q_t (the reward depends on the frozen generator’s sampling, not on the agent’s history at time t),

$$v_{t+1} = (1 - \alpha)^2 v_t + \alpha^2 \sigma^2, \quad \sigma^2 = \text{Var}(r_t | s).$$

The standard geometric-series expansion gives

$$v_{t+1} = (1 - \alpha)^{2(t+1)} v_0 + \alpha^2 \sigma^2 \sum_{k=0}^t (1 - \alpha)^{2k} \xrightarrow{t \rightarrow \infty} \frac{\alpha^2 \sigma^2}{1 - (1 - \alpha)^2} = \frac{\alpha}{2 - \alpha} \sigma^2,$$

which is the claimed bound. $\square$

5.4 Why Neither Baseline Closes the Effective-Learning Gap

We sketch the argument; the in-body summary is at the close of §1. MemRL’s retriever score is $(1 - \lambda)\text{sim} + \lambda\hat{Q}$ with $\lambda > 0$ [30]; when $Q_{\text{init}} = 0$ (a common default), a newly created skill contributes only the similarity term, so condition (c)

of §1 holds only when the embedding of the new skill is already close to the task embedding—exactly the cold-start trap. With *additive* Q , the boost is a positive externality for any high- Q skill regardless of similarity: a stale refined skill can out-rank a new relevant one, again violating (c). SkillQ’s *multiplicative* form $(1 + \beta\hat{Q}) \cdot \hat{\text{sim}}$ plus the hard threshold τ removes both externalities: a low-similarity skill is never rescued by a high Q , and a new skill with $\hat{\text{sim}} = 0$ cannot enter retrieval at all.

Memento-Skills’ frequency-ratio utility [12] faces the same externalities (utility added to similarity, not multiplied into it). For Defect 2: if the agent ignores a retrieved skill, the conditional $P(r_{\text{task}} \mid s)$ is unidentifiable from the observed outcome—no instrumental variable distinguishes “skill was called, task failed for environmental reasons” from “skill was not called”—so the expectation in Eq. (1) is undefined on this episode and the Q -update cannot be attributed. Mandatory-call makes the called skill well-defined on every retrieval event with r_{task} drawn from the skill-conditioned distribution, restoring identifiability with zero added hyperparameters; behaviour-cloning alternatives (per-token log-probability tracking, attention-based contribution) require per-token log-prob storage, are brittle to LLM sampling temperature and tool-use, and admit no closed-form variance bound.

Informal summary. Condition (c) is broken by any retriever that *adds* a value signal to similarity instead of *multiplying* it; condition (b) is broken by any attribution strategy that fails to make the called skill well-defined on every retrieval event. SkillQ’s two design choices—multiplicative retrieval plus mandatory-call—close both gaps with a single per-skill Q -value and a hard similarity threshold τ .

5.5 Preliminary Numerical Study of the Three Symptoms on TB 2.0

To make the two-defect claim in §1 falsifiable rather than verbal, we describe a small preliminary study on a stratified 30-task subset of TB 2.0 [13]: 10 tasks per difficulty tier (easy / medium / hard), seeds 0–4. The protocol mirrors §4 but uses a *no-mandatory-call* variant of MemRL [30] (denoted MEMRL-NOCALL) in which the agent may freely ignore a retrieved skill. We instrument three diagnostic quantities—the *stale-skill retrieval share* (S_1 , negation of condition (c): the fraction of retrieval calls whose top-1 result is a skill last improved more than 20 tasks earlier); the *mean first-retrieval round for a new skill* (S_2 , also a negation of (c): the mean over newly created skills of the first retrieval-call index at which the skill enters the top- k set); and the *Forgetting Rate* (FR) (S_3 , negation of condition (b): the fraction of tasks solved in epoch $e - 1$ that fail in epoch e , averaged across $e = 1, \dots, 5$).

The expected qualitative pattern (numbers to be reported in the camera-ready version): MEMRL ($S_1 = [\text{TBD}]$ %, mean first-retrieval round = $[\text{TBD}]$, $FR = [\text{TBD}]$ %) shows the joint signature of the two defects; MEMRL-NOCALL ($S_1 = [\text{TBD}]$ %, $FR = [\text{TBD}]$ %) isolates the contribution of Defect 2 by holding Defect 1 constant; **SkillQ** ($S_1 = [\text{TBD}]$ %, $FR = [\text{TBD}]$ %) breaks both signatures. The contrastive isolation is the preliminary analogue of the formal derivation of

§5.4: it shows that Defect 1 is necessary for S_1, S_2 and Defect 2 is necessary for S_3 .

5.6 LLM Prompt for Environment-Failure Detection

The `FAIL_ENV_ISSUE` Enum is the only attribution in Table 3 whose classification is delegated to a learned signal rather than the deterministic verifier. The classification is performed by the agent’s own LLM immediately after the task verdict, using the following prompt.

*You are a failure-attribution classifier. You will see a task description, the trajectory the agent just executed, the deterministic verifier verdict (**pass** or **fail**), and the retrieved skill (if any). Decide whether the failure is attributable to the skill’s content or to the environment.*

*Return **ENV_ISSUE** if any of the following holds:*

- the verifier timed out (the task environment hung);*
- the task specification itself is malformed (e.g. a referenced file does not exist on disk, an external API the task depends on is unreachable, a Docker container failed to start);*
- the failure reproduces on a fresh agent run without any skill (i.e. the task is unsolvable in this environment);*
- the verifier’s own check is broken (e.g. the expected output file is the wrong path, the test is comparing against a stale snapshot).*

*Return **MODEL_ISSUE** otherwise (the agent’s reasoning, the skill’s content, or the agent’s tool-use is at fault).*

*Reply with one line: **ENV_ISSUE** or **MODEL_ISSUE**.*

The classification is logged to `q_updates.jsonl` as a `env_flag` field but does not propagate to the Q -update (Table 3, row 5). The masked-transition argument of §3.4 is the formal justification for this design choice.

5.7 Edge-Case Rule Table

The attribution layer must classify *every* task outcome into exactly one of the five Enums of Table 3, without ambiguity. Table 7 lists the four edge cases that arise in practice and the rule that resolves each.

These rules are deterministic and require no learned classifier; only `FAIL_ENV_ISSUE` (Appendix 5.6) involves a learned decision. The full content-action mapping is given in Table 3; the present table is a complement that resolves the boundary cases of the mapping.

Observable	Rule	Rationale
Verifier returned pass but the trajectory contains an explicit error log line.	Classify SUCCESS_SKILL_USED SUCCESS_NO_SKILL_SEEN do <i>not</i> flip to failure.	as The deterministic verifier (or is authoritative; agent self-explicit error log line); doubt is not.
Half-completion: 3 of 5 required sub-tasks passed.	Classify FAILURE_SKILL_USED FAILURE_SKILL_NOT_USED	as Verifier is binary; partial (or credit is not a reward, it is an opportunity for the next retrieval's mechanism to surface a better skill.
Verifier is itself broken (e.g. wrong expected output path).	Classify FAIL_ENV_ISSUE	as The skill's quality is undefined for this episode; the masked-transition argument applies.
Retrieved top- k is empty (no skill cleared τ) and the agent succeeded by its own capability.	Classify SUCCESS_NO_SKILL_SEEN trigger content action <i>create new</i> .	as The trajectory is a positive example of procedural knowledge that was not in the library; capturing it prevents the library from missing a recurring pattern.

Table 7. Edge-case rules for the 5-Enum attribution. Each row lists the observable condition, the rule that maps it to an Enum, and the rationale. Half-completions are folded into the failure branch; partial credit is delivered through the next retrieval's $\hat{\text{sim}}(1 + \beta \widehat{Q(s)}) + \gamma$ UCB mechanism rather than a fractional reward.